

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS COURSE CODE: CSA0711

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period.

Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements ; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem

Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, wellorganized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1. PYTHON PROGRAM TO GENERATE A STAR TOPOLOGY

1.1 Aim

To develop a Python program that generates a Star Topology for a given number of hosts connected to a

central switch. The program accepts the number of hosts as input and displays the topology in ASCII format.

The program also displays:

- Host label

- Central switch name

- Cat6 UTP cable length between each host and switch

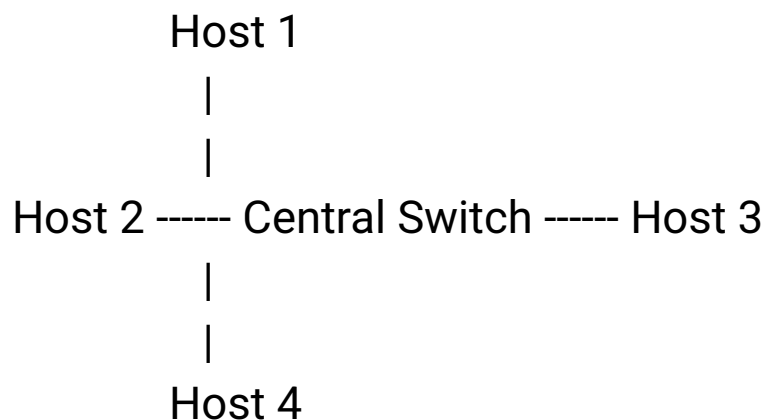
- Connectivity verification

1.2 Introduction

A Star Topology is a network topology in which all devices are directly connected to a central networking device, usually a switch.

In this topology, the central switch acts as the main communication point. Each host has a separate connection to the switch.

For example:



If there are N hosts, the star topology requires N individual connections from the hosts to the central switch.

1.3 Python Program

Program to Generate Star Topology

```

def create_star_topology(num_hosts):

    switch = "Central-Switch"

    print("\n===== STAR TOPOLOGY =====")
    print("          [", switch, "]")
    print("          |")

    connections = []

    # Create connections for each host
    for i in range(1, num_hosts + 1):

        host = f"Host-{i}"

        # Example Cat6 cable length
        cable_length = 10 + (i * 5)

        connections.append((host, switch, cable_length))

    # Display topology
    for host, switch, length in connections:
        print(f"{host:<10} ---- Cat6 UTP ({length} m) ----
{switch}")

    # Verify connectivity
    print("\n===== CONNECTIVITY CHECK
=====")

```

```

if len(connections) == num_hosts:
    print("All hosts are connected through the central
switch.")
    print("Total Hosts:", num_hosts)
    print("Total Connections:", len(connections))
else:
    print("Connectivity verification failed.")

```

Get number of hosts

```
num_hosts = int(input("Enter the number of hosts: "))
```

```
if num_hosts <= 0:
```

```
    print("Number of hosts must be greater than zero.")
```

```
else:
```

```
    create_star_topology(num_hosts)
```

1.4 Sample Input

Enter the number of hosts: 4

1.5 Sample Output

===== STAR TOPOLOGY =====

[Central-Switch]

|

Host-1 ---- Cat6 UTP (15 m) ---- Central-Switch

Host-2 ---- Cat6 UTP (20 m) ---- Central-Switch

Host-3 ---- Cat6 UTP (25 m) ---- Central-Switch

Host-4 ---- Cat6 UTP (30 m) ---- Central-Switch

===== CONNECTIVITY CHECK =====

All hosts are connected through the central switch.

Total Hosts: 4

Total Connections: 4

1.6 Program Explanation

Step 1 – Input

The program asks the user to enter the number of hosts.

```
num_hosts = int(input("Enter the number of hosts: "))
```

Step 2 – Central Switch

A central switch is created with the name:

```
switch = "Central-Switch"
```

Step 3 – Host Creation

A for loop creates each host.

For example:

Host-1

Host-2

Host-3

Host-4

Step 4 – Cable Length

Each host is assigned a simulated Cat6 UTP cable length.

The length is stored along with the host and switch information.

Step 5 – Connectivity Verification

The program compares the number of created connections with the number of hosts.

If both values are equal, it confirms that all hosts are

connected.

1.7 How Star Topology Provides Reliable Communication

Star topology provides reliable LAN communication because each host has an independent connection to the central switch.

Advantages

1. Easy fault detection – If one cable fails, only that host is disconnected.
2. Independent connections – Failure of one connection does not normally affect other hosts.
3. Easy management – All connections are controlled through the central switch.
4. Easy expansion – New hosts can be added by connecting another port.
5. Better performance – A switch can provide dedicated communication paths between devices.
6. Simple troubleshooting – Network administrators can identify individual faulty ports or cables.

Disadvantage

The main disadvantage is that the central switch is a critical point. If the central switch fails, communication between the connected hosts is disrupted.

1.8 Conclusion

The Python program successfully generates a star topology for a user-defined number of hosts. Each host is connected independently to the central switch using a simulated Cat6 UTP cable. The program also verifies that all hosts have

connections to the central switch. Star topology is widely used in LANs because it provides easy management, simple troubleshooting, scalability, and reliable communication.

2. PYTHON FUNCTION TO VALIDATE UTP CABLE LENGTH

2.1 Aim

To develop a Python function named `validate_segment(length_m)` that checks whether a UTP cable segment satisfies the 100-meter maximum segment length specified for standard Ethernet copper cabling.

The function should:

- Accept cable length in meters.

- Return True for a valid length.

- Return False with an appropriate error message for an invalid length.

- Test the function using at least three different values.

2.2 Python Program

UTP Cable Length Validation

```
def validate_segment(length_m):
```

```
    MAX_LENGTH = 100
```

```
    if length_m <= 0:
```

```
        print("Error: Cable length must be greater than 0  
meters.")
```

```
        return False
```

```
if length_m <= MAX_LENGTH:
    print(f"{length_m} m: Valid UTP cable segment.")
    return True

else:
    print(
        f"{length_m} m: Invalid UTP cable segment. "
        f"Maximum permitted length is {MAX_LENGTH}
meters."
    )
    return False
```

Test Case 1

```
result1 = validate_segment(50)
print("Result:", result1)
```

Test Case 2

```
result2 = validate_segment(100)
print("Result:", result2)
```

Test Case 3

```
result3 = validate_segment(120)
print("Result:", result3)
```

2.3 Sample Output

50 m: Valid UTP cable segment.

Result: True

100 m: Valid UTP cable segment.

Result: True

120 m: Invalid UTP cable segment. Maximum permitted length is 100 meters.

Result: False

2.4 Test Case Analysis

Test Case	Cable Length	Expected Result	Actual Result
1	50 m	Valid	True
2	100 m	Valid	True
3	120 m	Invalid	False

Test Case 1 – 50 meters

The cable is less than the 100-meter limit.

Therefore:

$$50 \leq 100$$

Result:

True

Test Case 2 – 100 meters

The cable is exactly at the maximum permitted length.

Therefore:

$$100 \leq 100$$

Result:

True

Test Case 3 – 120 meters

The cable exceeds the maximum permitted length.

Therefore:

$120 > 100$

Result:

False

2.5 Additional Validation

The program also checks whether the cable length is zero or negative.

For example:

`validate_segment(0)`

Output:

Error: Cable length must be greater than 0 meters.

This prevents invalid cable length values from being accepted.

2.6 Importance of Cable Length Validation

Cable length is important in Ethernet networks because excessive copper-cable distance can result in signal attenuation and unreliable communication.

Validation helps:

- Prevent incorrect network design.

- Ensure cable segments remain within the specified limit.

- Reduce communication problems.

- Improve network reliability.

- Support proper network planning.

2.7 Conclusion

The `validate_segment(length_m)` function successfully

checks whether a UTP cable segment is within the 100-meter maximum. The three test cases demonstrate that 50 m and 100 m are accepted, while 120 m is rejected. Therefore, the function provides a simple method for validating Ethernet cable segment lengths.

3. PYTHON SIMULATION OF CSMA/CD IN A STAR TOPOLOGY

3.1 Aim

To develop a Python program that simulates the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol for four hosts connected through a central switch.

The program should:

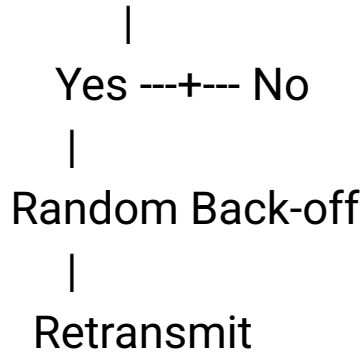
- Simulate frame transmission.
- Detect collisions.
- Generate random back-off intervals.
- Continue transmission after waiting.
- Record transmission attempts.
- Record collision events.
- Record back-off intervals.
- Store the events in `csma_log.txt`.

3.2 Introduction

CSMA/CD is a medium-access technique historically associated with shared, half-duplex Ethernet. A transmitting device first senses whether the medium is available. If two devices transmit at the same time, a collision may occur.

The general process is:

Sense → Transmit → Detect Collision



Important note: In modern switched full-duplex Ethernet, collisions normally do not occur and CSMA/CD is not used. This program is therefore an educational simulation of the protocol behavior.

3.3 Python Program

```
import random
import time
```

```
HOSTS = ["Host-1", "Host-2", "Host-3", "Host-4"]
MAX_ATTEMPTS = 5
```

```
LOG_FILE = "csma_log.txt"
```

```
def log_message(message):
    with open(LOG_FILE, "a") as file:
        file.write(message + "\n")
```

```
def transmit_frame(host):
```

```
    attempt = 0
```

```
while attempt < MAX_ATTEMPTS:
```

```
    attempt += 1
```

```
    message = (  
        f"{host}: Transmission attempt {attempt}"  
    )
```

```
    print(message)  
    log_message(message)
```

```
    # Simulate whether another host transmits  
    collision = random.choice([True, False])
```

```
    if collision:
```

```
        message = f"{host}: COLLISION detected."  
        print(message)  
        log_message(message)
```

```
        # Binary exponential backoff  
        backoff_slots = random.randint(  
            0, (2 ** attempt) - 1  
        )
```

```
        message = (  
            f"{host}: Back-off interval = "  
            f"{backoff_slots} time slots."
```

```

    )

    print(message)
    log_message(message)

    # Simulated waiting
    time.sleep(backoff_slots * 0.1)

    log_message(
        f"{host}: Retransmitting after back-off."
    )

else:

    message = (
        f"{host}: Frame transmitted successfully."
    )

    print(message)
    log_message(message)

    return True

message = (
    f"{host}: Maximum attempts reached. "
    f"Transmission failed."
)

print(message)

```

```
log_message(message)
```

```
return False
```

```
# Start a new log
```

```
with open(LOG_FILE, "w") as file:
```

```
    file.write("==== CSMA/CD TRANSMISSION LOG  
====\n")
```

```
print("==== CSMA/CD SIMULATION ====")
```

```
for host in HOSTS:
```

```
    transmit_frame(host)
```

```
print("\nSimulation completed.")
```

```
print("Events are recorded in csma_log.txt")
```

3.4 Sample Console Output

Because collisions and back-off values are randomly generated, the exact output can change each time.

Example:

```
==== CSMA/CD SIMULATION ====
```

```
Host-1: Transmission attempt 1
```

```
Host-1: COLLISION detected.
```

```
Host-1: Back-off interval = 1 time slots.
```

```
Host-1: Transmission attempt 2
```

Host-1: Frame transmitted successfully.

Host-2: Transmission attempt 1

Host-2: Frame transmitted successfully.

Host-3: Transmission attempt 1

Host-3: COLLISION detected.

Host-3: Back-off interval = 2 time slots.

Host-3: Transmission attempt 2

Host-3: Frame transmitted successfully.

Host-4: Transmission attempt 1

Host-4: Frame transmitted successfully.

Simulation completed.

Events are recorded in csma_log.txt

3.5 Log File

The program creates:

csma_log.txt

Example contents:

===== CSMA/CD TRANSMISSION LOG =====

Host-1: Transmission attempt 1

Host-1: COLLISION detected.

Host-1: Back-off interval = 1 time slots.

Host-1: Retransmitting after back-off.

Host-1: Transmission attempt 2

Host-1: Frame transmitted successfully.

Host-2: Transmission attempt 1

Host-2: Frame transmitted successfully.
Host-3: Transmission attempt 1
Host-3: COLLISION detected.
Host-3: Back-off interval = 2 time slots.
Host-3: Retransmitting after back-off.
Host-3: Transmission attempt 2
Host-3: Frame transmitted successfully.

3.6 Program Explanation

Step 1 – Define Hosts

Four hosts are defined:

```
HOSTS = ["Host-1", "Host-2", "Host-3", "Host-4"]
```

Step 2 – Transmission Attempt

Each host attempts to transmit a frame.

The attempt number is increased each time.

Step 3 – Collision Detection

The program randomly determines whether a collision occurs.

```
collision = random.choice([True, False])
```

This is used only to simulate the behavior.

Step 4 – Back-Off

When a collision occurs, the host calculates a random back-off interval.

The simulation uses a simplified form of binary exponential backoff:

```
random.randint(0, (2 ** attempt) - 1)
```

The host waits for the generated number of time slots before attempting transmission again.

Step 5 – Successful Transmission

If no collision occurs:

Frame transmitted successfully.

The event is written into the log file.

Step 6 – Maximum Attempts

The program limits the number of attempts to prevent an infinite loop.

Here:

MAX_ATTEMPTS = 5

3.7 How CSMA/CD Reduces Transmission Conflicts

CSMA/CD reduces conflicts through several steps.

Carrier Sensing

A host checks whether the shared medium is being used before transmitting.

Collision Detection

If two transmissions overlap, the collision is detected.

Transmission Stopping

The transmitting devices stop the unsuccessful transmission attempt.

Random Back-Off

Each device waits for a randomly selected period.

This reduces the probability that all devices will immediately retransmit at the same time.

Retransmission

After the waiting period, the host attempts to transmit again.

3.8 Advantages

1. Detects transmission collisions.

2. Uses random back-off.
3. Allows retransmission after collision.
4. Records network events.
5. Helps demonstrate Ethernet medium-access behavior.
6. Provides useful information for analyzing collision frequency.

3.9 Conclusion

The Python program successfully simulates CSMA/CD behavior for four hosts. It generates transmission attempts, detects simulated collisions, calculates random back-off intervals, and retries transmission. Every important event is stored in `csma_log.txt`.

The simulation demonstrates how collision detection and randomized back-off reduce repeated transmission conflicts. In practical modern switched full-duplex Ethernet, however, collisions are normally eliminated and CSMA/CD is not required.

4. PYTHON CLIENT-SERVER APPLICATION WITH MAC ADDRESS LEARNING

4.1 Aim

To develop a Python client-server application using the socket library to simulate communication between two hosts connected through a central switch.

The server will:

- Receive frames from the client.
- Simulate forwarding.

Maintain a MAC address table.
Learn MAC addresses.
Associate MAC addresses with ports.
Display the updated MAC address table after successful transmission.

4.2 Introduction

An Ethernet switch maintains a MAC address table to determine where frames should be forwarded.

For example:

MAC Address	Port
AA:AA:AA:AA:AA:01	Port 1
BB:BB:BB:BB:BB:02	Port 2

When a frame arrives, the switch learns the source MAC address and associates it with the incoming port.

If the destination MAC address is already present in the table, the switch forwards the frame only through the corresponding port.

4.3 Server Program

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
# Simulated MAC address table
```

```
mac_table = {}
```

```

def display_mac_table():

    print("\n===== MAC ADDRESS TABLE =====")

    if not mac_table:
        print("MAC table is empty.")
        return

    print("MAC Address          Port")

    for mac, port in mac_table.items():
        print(f"{mac:<27} {port}")


# Create server socket
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

server_socket.bind((HOST, PORT))
server_socket.listen(1)

print("Switch Server is waiting for connection...")

connection, address = server_socket.accept()

```

```
print("Client connected:", address)
```

```
while True:
```

```
    data = connection.recv(1024)
```

```
    if not data:
```

```
        break
```

```
    message = data.decode()
```

```
    # Frame format:
```

```
    # source_mac,destination_mac,message
```

```
    parts = message.split(",", 2)
```

```
    if len(parts) != 3:
```

```
        print("Invalid frame received.")
```

```
        continue
```

```
    source_mac = parts[0]
```

```
    destination_mac = parts[1]
```

```
    payload = parts[2]
```

```
    # Learn source MAC address
```

```
    mac_table[source_mac] = "Port-1"
```

```
    print("\nFrame received")
```

```
    print("Source MAC:", source_mac)
```

```
    print("Destination MAC:", destination_mac)
```

```
print("Data:", payload)

# Forwarding decision
if destination_mac in mac_table:

    destination_port = mac_table[destination_mac]

    print(
        "Destination found in MAC table."
    )

    print(
        "Forwarding frame to:",
        destination_port
    )

else:

    print(
        "Destination MAC not found."
    )

    print(
        "Frame would be flooded to other ports."
    )

# Send acknowledgement
connection.send(
    b"Frame successfully processed by switch."
```

)

display_mac_table()

connection.close()

server_socket.close()

print("Server stopped.")

4.4 Client Program

import socket

HOST = "127.0.0.1"

PORT = 5000

```
client_socket = socket.socket(  
    socket.AF_INET,  
    socket.SOCK_STREAM  
)
```

client_socket.connect((HOST, PORT))

source_mac = "AA:AA:AA:AA:AA:01"

destination_mac = "BB:BB:BB:BB:BB:02"

message = "Hello from Host-1"

frame = (

```
    source_mac
    + ","
    + destination_mac
    + ","
    + message
)

client_socket.send(frame.encode())

response = client_socket.recv(1024)

print("Server:", response.decode())

client_socket.close()
```

4.5 Sample Output

Server

Switch Server is waiting for connection...

Client connected: ('127.0.0.1', 54321)

Frame received

Source MAC: AA:AA:AA:AA:AA:01

Destination MAC: BB:BB:BB:BB:BB:02

Data: Hello from Host-1

Destination MAC not found.

Frame would be flooded to other ports.

===== MAC ADDRESS TABLE =====

MAC Address	Port
AA:AA:AA:AA:AA:01	Port-1

The switch has learned the source MAC address.

4.6 MAC Learning Process

When a switch receives a frame, it examines the source MAC address.

For example:

Source MAC = AA:AA:AA:AA:AA:01

Incoming Port = Port-1

The switch stores:

AA:AA:AA:AA:AA:01 → Port-1

This is called MAC address learning.

4.7 Frame Forwarding

Suppose the switch has learned:

AA:AA:AA:AA:AA:01 → Port-1

BB:BB:BB:BB:BB:02 → Port-2

If Host 1 sends a frame to Host 2:

Source = AA:AA:AA:AA:AA:01

Destination = BB:BB:BB:BB:BB:02

The switch checks the MAC table.

It finds:

BB:BB:BB:BB:BB:02 → Port-2

Therefore, it forwards the frame through Port-2.

4.8 Unknown Destination

If the destination MAC address is not present in the table, the switch cannot determine the destination port.

Therefore, the frame is flooded to appropriate other ports within the LAN/VLAN.

Once the destination host responds, the switch can learn its MAC address from the source address of that response.

4.9 Advantages

1. Reduces unnecessary network traffic.
2. Learns device locations automatically.
3. Enables efficient frame forwarding.
4. Supports dynamic network operation.
5. Allows the MAC table to change as devices move or communicate.

4.10 Conclusion

The client-server Python application demonstrates the basic concept of Ethernet switching. The simulated switch learns source MAC addresses and associates them with ports. When the destination MAC address is known, the frame can be forwarded to the appropriate port. If the destination is unknown, the frame is flooded. This demonstrates the basic principles of MAC learning and Ethernet frame forwarding.

5. PYTHON CLASS TO MODEL AN ETHERNET STAR SWITCH

5.1 Aim

To design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch in a star topology.

The class should provide methods to:

- Add a host/port.

- Remove a host/port.

- Broadcast a frame.

- Maintain active ports.

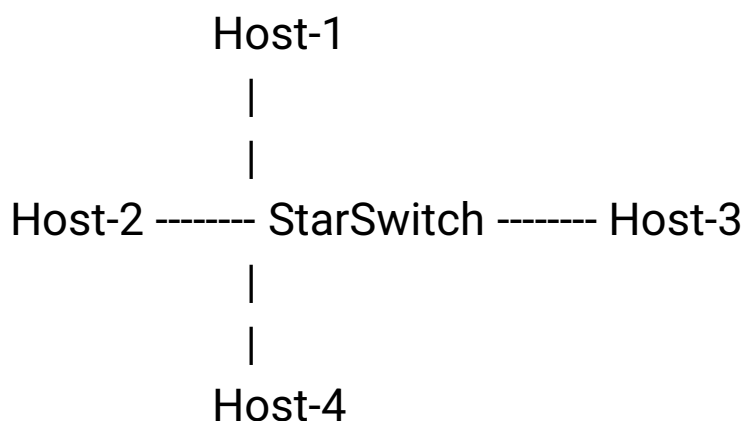
- Log broadcast operations.

- Display the source and destination hosts receiving the frame.

5.2 Introduction

In a star topology, the central switch connects all hosts.

For example:



When a frame needs to be broadcast, the switch sends it to all connected ports except the source port.

5.3 Python Program

class StarSwitch:

```
def __init__(self):
    self.active_ports = []
    self.broadcast_log = []
```

```
# Add a host
def add_port(self, host_id):

    if host_id not in self.active_ports:

        self.active_ports.append(host_id)

        print(
            f"{host_id} connected to the switch."
        )

    else:

        print(
            f"{host_id} is already connected."
        )

# Remove a host
def remove_port(self, host_id):

    if host_id in self.active_ports:

        self.active_ports.remove(host_id)

        print(
            f"{host_id} disconnected from the switch."
        )

    else:
```

```
print(  
    f"{host_id} is not connected."  
)
```

Broadcast a frame

```
def broadcast(self, frame, source):
```

```
    print("\n==== BROADCAST OPERATION ====")
```

```
    if source not in self.active_ports:
```

```
        print(  
            "Error: Source host is not connected."  
        )
```

```
        return
```

```
    receivers = []
```

```
    # Send to all ports except source
```

```
    for host in self.active_ports:
```

```
        if host != source:
```

```
            receivers.append(host)
```

```
            print(  
                f"Frame '{frame}' sent from "
```

```

        f"{source} to {host}"
    )

    # Store broadcast operation
    log_entry = {
        "frame": frame,
        "source": source,
        "destinations": receivers
    }

    self.broadcast_log.append(log_entry)

    print(
        "Broadcast operation completed."
    )

    # Display active ports
    def show_ports(self):

        print("\n===== ACTIVE PORTS =====")

        if len(self.active_ports) == 0:

            print("No active ports.")

        else:

            for host in self.active_ports:

```

```
print(host)
```

```
# Display broadcast log
```

```
def show_log(self):
```

```
    print("\n===== BROADCAST LOG =====")
```

```
    for entry in self.broadcast_log:
```

```
        print(
            f"Source: {entry['source']}"
        )
```

```
        print(
            f"Frame: {entry['frame']}"
        )
```

```
        print(
            f"Destinations: "
            f"{', '.join(entry['destinations'])}"
        )
```

```
# Create switch
```

```
switch = StarSwitch()
```

```
# Add multiple hosts
```

```
switch.add_port("Host-1")
```

```
switch.add_port("Host-2")
```

```
switch.add_port("Host-3")
switch.add_port("Host-4")
```

```
# Display active ports
switch.show_ports()
```

```
# Broadcast a frame
switch.broadcast(
    "Network Announcement",
    "Host-1"
)
```

```
# Remove a host
switch.remove_port("Host-4")
```

```
# Display active ports again
switch.show_ports()
```

```
# Another broadcast
switch.broadcast(
    "Second Message",
    "Host-2"
)
```

```
# Display log
switch.show_log()
```

5.4 Sample Output

Host-1 connected to the switch.

Host-2 connected to the switch.

Host-3 connected to the switch.

Host-4 connected to the switch.

===== ACTIVE PORTS =====

Host-1

Host-2

Host-3

Host-4

===== BROADCAST OPERATION =====

Frame 'Network Announcement' sent from Host-1 to Host-2

Frame 'Network Announcement' sent from Host-1 to Host-3

Frame 'Network Announcement' sent from Host-1 to Host-4

Broadcast operation completed.

Host-4 disconnected from the switch.

===== ACTIVE PORTS =====

Host-1

Host-2

Host-3

===== BROADCAST OPERATION =====

Frame 'Second Message' sent from Host-2 to Host-1

Frame 'Second Message' sent from Host-2 to Host-3

Broadcast operation completed.

===== BROADCAST LOG =====

Source: Host-1

Frame: Network Announcement

Destinations: Host-2, Host-3, Host-4

Source: Host-2

Frame: Second Message

Destinations: Host-1, Host-3

5.5 Program Explanation

`__init__()`

The constructor initializes the switch.

```
self.active_ports = []
```

```
self.broadcast_log = []
```

`active_ports` stores currently connected hosts.

`broadcast_log` stores information about previous broadcast operations.

`add_port()`

This method connects a host to the switch.

```
switch.add_port("Host-1")
```

The host is added to the active-port list.

`remove_port()`

This method disconnects a host.

```
switch.remove_port("Host-4")
```

The host is removed from the active-port list.

`broadcast()`

This method sends a frame to all active hosts except the

source.

For example:

Source = Host-1

Active hosts:

Host-1

Host-2

Host-3

Host-4

The frame is sent to:

Host-2

Host-3

Host-4

It is not sent back to Host-1.

5.6 Significance of Broadcasting

Broadcasting is an important communication mechanism in Ethernet networks.

A broadcast frame is intended for all appropriate devices in the same broadcast domain.

Examples include network operations such as:

- ARP requests

- DHCP discovery

- Network announcements

For example, when a device needs to discover the MAC address associated with an IPv4 address, an ARP request is normally broadcast within the local broadcast domain.

5.7 Advantages of Broadcasting

1. Device Discovery

Broadcasting can help a device discover information about other devices on the local network.

2. Network Configuration

Protocols such as DHCP use broadcast communication during parts of the address-assignment process.

3. Simple Communication

The sender does not need to know every destination individually when the message is intended for the whole broadcast domain.

4. Network Management

Broadcast messages can support certain network-management and discovery functions.

5.8 Limitation of Broadcasting

Excessive broadcast traffic can consume network resources and affect performance.

Therefore, networks use mechanisms such as:

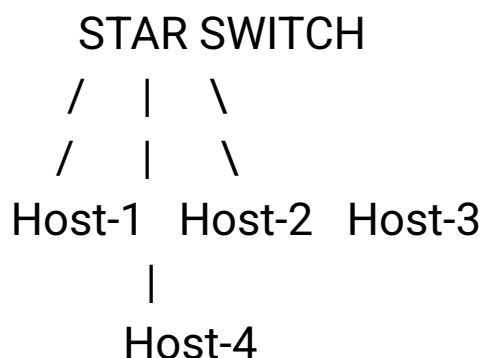
- VLANs

- Routers

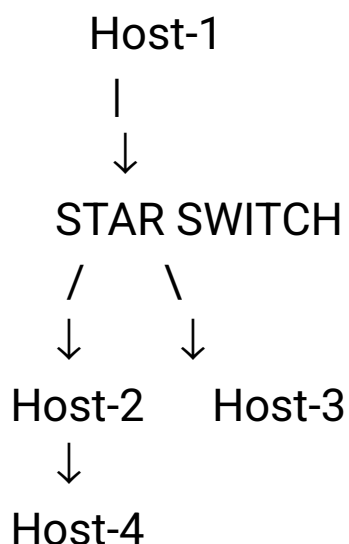
- Broadcast-domain segmentation

to control the size of broadcast domains.

5.9 Overall Working



When Host-1 sends a broadcast:



The source host is excluded from receiving its own broadcast.

5.10 Conclusion

The StarSwitch Python class successfully models basic Ethernet switch functionality. It maintains a list of active hosts, supports adding and removing ports, broadcasts frames to all connected hosts except the source, and records every broadcast operation.

The program demonstrates how a central switch can manage communication between multiple hosts in a star topology. Broadcasting is useful for local network discovery and configuration, although excessive broadcast traffic should be controlled through appropriate network segmentation.

OVERALL CONCLUSION

These five coding challenges demonstrate important concepts in computer networking and Python programming.

1. Star Topology Generation demonstrates how hosts can be connected to a central switch and how connectivity can be verified programmatically.
2. UTP Cable Validation demonstrates input validation and compliance with the maximum Ethernet copper-segment length.
3. CSMA/CD Simulation demonstrates collision detection, random back-off, retransmission, and event logging in a simplified shared-Ethernet environment.
4. Client-Server MAC Learning demonstrates how switches learn MAC addresses and use the MAC table to make forwarding decisions.
5. StarSwitch Class demonstrates object-oriented programming while modeling active switch ports and broadcast operations.

Together, these programs show how Python can be used to model, simulate, and analyze networking concepts, making it easier to understand the behavior of Ethernet networks, switches, transmission protocols, and network infrastructure.